Computer Security Symposium 2026
19-23 October 2026

Ghidra High P-codeに基づく関数の意味的等価性判定

斎藤 勇斗[1,a)]　掛井 将平[1]　齋藤 彰一[1]

概要：バイナリレベルの OSS/TPL 検出は，脆弱性・ライセンス・サプライチェーンリスク把握に重要である．既存検出手法は類似関数候補の生成と集約によりライブラリを判定する．しかし，構造が類似するだけで出力が異なる候補関数によりスコアが過大評価され，false positive を招く．候補生成や集約の精度向上は進む一方，個々の候補の動作面での信頼性検証は不十分である．本研究では，関数対応候補の集約前に再検証する後段手法を提案する．Ghidra の High P-code から戻り値と出力ポインタのデータフローを抽出し，同一入力での出力差の有無を SMT ソルバで検査する．評価では，同一ソースを異なるコンパイラ・最適化・アーキテクチャで生成した関数の同一性と意味を変えた関数の非同一性を判定し，さらに未判定の場合の要因と処理時間を確認する．以上により，後段検証器としての判定能力と限界を明らかにする．

キーワード：バイナリ解析，プログラム等価性，High P-code，SMT

Determining Function Semantic Equivalence Using Ghidra High P-Code

Hayato Saito[1,a)]　Shohei Kakei[1]　Shoichi Saito[1]

Abstract: Binary-level detection of Open-source software and Third-Party Libraries is important for assessing vulnerabilities, license compliance, and software supply-chain risks. Existing detection methods identify libraries by generating similar-function candidates and aggregating them at the library level. However, candidate functions that are structurally similar but produce different outputs may overestimate detection scores and lead to false positives. Although the accuracy of candidate generation and aggregation has improved, the behavioral reliability of individual candidates has not been sufficiently verified. This study proposes a post-verification method that re-examines function correspondence candidates before aggregation. The method extracts data flows related to return values and output pointers from Ghidra High P-code, and uses an SMT solver to check whether the outputs differ for the same input. In the evaluation, we examine whether the method can determine the equivalence of functions generated from the same source code under different compilers, optimization levels, and architectures, as well as the non-equivalence of functions with modified semantics. We also analyze the causes of inconclusive results and the processing time. Through this evaluation, we clarify the decision capability and limitations of the proposed method as a post-verification component.

Keywords: binary analysis, program equivalence, High P-code, SMT

1. はじめに

ソフトウェアに含まれる Open-source software（OSS）および Third-Party Library（TPL）をバイナリから特定する技術は，既知脆弱性の影響調査，ライセンス管理，Software Bill of Materials（SBOM）の作成などに用いられる．バイナリを対象とした TPL 検出は，大きく前段と後段の 2 段階からなる [1]．前段では，対象バイナリの各関数と既知ライブラリの関数との類似度を求め，類似度の高い関数対を関数対応候補として抽出する．後段では，得られた関数対応候補をライブラリ単位に集約し，TPL の有無を判定する．

[1] 名古屋工業大学
Nagoya Institute of Technology
[a)] h.saito.652@stn.nitech.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

既存手法の多くは，この前段，すなわち候補生成を対象とする．命令列，制御構造，意味的特徴，関数間の呼出し関係などを用いて，多数の既知関数から候補を探索する．しかし，これらの手法が根拠とするのは特徴や構造の類似性であり，候補となった 2 つの関数が同じ入力に対して同じ動作をすることまでは保証しない．動作の異なる関数対が候補に含まれたまま後段の集約で TPL 判定の根拠とされると，誤った TPL 判定を招く．

そこで本研究では，前段が生成した関数対応候補を入力として受け取り，後段でライブラリ単位へ集約する前に，入出力の等価性に基づいて検証する後段検証器を提案する．Ghidra が機械語から復元する中間表現 High P-code と Satisfiability Modulo Theories（SMT）ソルバ Z3 を用い，両関数の入力を対応づけた上で，戻り値と出力ポインタを介した書き込みが一致するかを判定する．

評価には，等価・非等価なプログラム対を収録するデータセット EqBench[2] を用いる．同一 C ソースを異なる ISA，コンパイラ，最適化レベルでビルドした関数対に対して等価と判定できる割合と，ソースコードレベルで非等価な関数対に対して非等価と判定できる割合を評価し，判定結果とプログラム特徴との関連，および代表的な判定事例を分析する．本研究の評価対象は，候補集約前の後段検証器としての判定能力と限界である．

本稿の構成は次の通りである．2 章で関連研究を整理し，本研究の位置づけを述べる．3 章で提案手法を述べる．4 章で EqBench を用いた評価と考察を述べる．5 章でまとめと今後の課題を述べる．

2. 背景と関連研究

2.1 バイナリレベルの TPL 検出

バイナリレベルの TPL 検出は，対象バイナリから文字列，命令，関数の意味的特徴，関数呼出し関係などを抽出し，既知ライブラリとの対応を求める．ModX[3] は意味的特徴とプログラムのモジュール化を組み合わせて部分的に取り込まれた TPL を検出し，BinaryAI[1] は関数の意味的表現とリンク時の局所性を用いて候補を絞り込む．これらの手法は，コンパイラや最適化による表現差に対して候補を探索し，得られた候補をライブラリ単位へ集約するが，候補ごとの入出力等価性は判定条件に含まれない．

2.2 バイナリの意味比較

BinHunt[4] は基本ブロックをシンボリック実行し，意味的に対応するブロックと制御フローから関数の類似性を評価する．iBinHunt[5] は BinHunt の手法を関数間制御フローの差分探索へ拡張した．いずれもバイナリ全体または関数全体を対象に意味的な差分を探索する手法であり，既に候補として与えられた関数対を検証する後段検証器としては設計されていない．

BinGo[6] はクロスアーキテクチャでの類似関数検索を扱い，BLEX[7] と BinSim[8] は実行またはトレースに基づく意味比較を行う．これらに対し，本研究は High P-code と SMT ソルバを用いて，実行環境なしに入出力の対応関係を検証する．

DDEC[9] は，x86 の 2 つのループが同じ動作をするかを検証する手法であり，実行結果から対応関係の候補を求め，SMT ソルバで検証する．低水準コードの意味的一致を検証する手法も提案されている [10]．これらはループを含むプログラムの等価性を扱う点で本研究と共通するが，対象は個別のループやプログラム断片であり，TPL 検出において前段が生成した多数の関数対応候補を，後段の集約に先立って検証する枠組みではない．

High P-code の形式意味論は既存研究で言及されている [11]．これは等価性判定手法ではないが，本研究が High P-code を解析対象とする際の意味論的な裏付けとなる．

2.3 本研究の位置づけ

本研究は，前段が生成した関数対応候補を入力とし，ライブラリ単位の集約前に対応入力に対する観測結果を比較する後段検証器として位置づけられる．

3. 提案手法

3.1 処理の全体像

提案手法は前段と後段の中間に位置し，前段が生成した関数対応候補を比較対象のバイナリ関数対として受け取り，ライブラリ単位の集約前に意味的に再検証する．**図 1** にこの位置づけと処理の概要を示す．図 1 の各ノード内の数字は関連する節番号を示す。

各関数から Ghidra の中間表現 High P-code と Control Flow Graph（CFG）を取得し，戻り値と出力ポインタへの書き込みを観測結果として入力と対応づける．関数入口から終了点までを終了経路，その経路を通る入力条件を経路条件，観測結果を入力から表す式を観測式とする．両関数の終了経路を 1 つずつ選んだ組を経路対，両経路の観測結果の差を表す条件を不一致条件とし，SMT ソルバ Z3 で充足可能性を判定する．

各経路対について確認結果と確認できなかった事項を記録する．全ての経路対を処理した後，追加情報が得られる場合は，必要に応じて Ghidra の関数解析情報を更新し，制約を再構成して判定を再試行する．最後に，全ての経路対の結果を集約して関数対応候補の扱いに反映し，後段でライブラリ単位に集約して TPL の有無を判定する．

3.2 前提と入出力

提案手法への入力は，関数対応候補，調査対象バイナリ，および既知 TPL からビルドした参照バイナリである．比較する関数対を $F = (f_1, f_2)$ とし，f_1 を調査対象バイナリ

© 2026 Information Processing Society of Japan

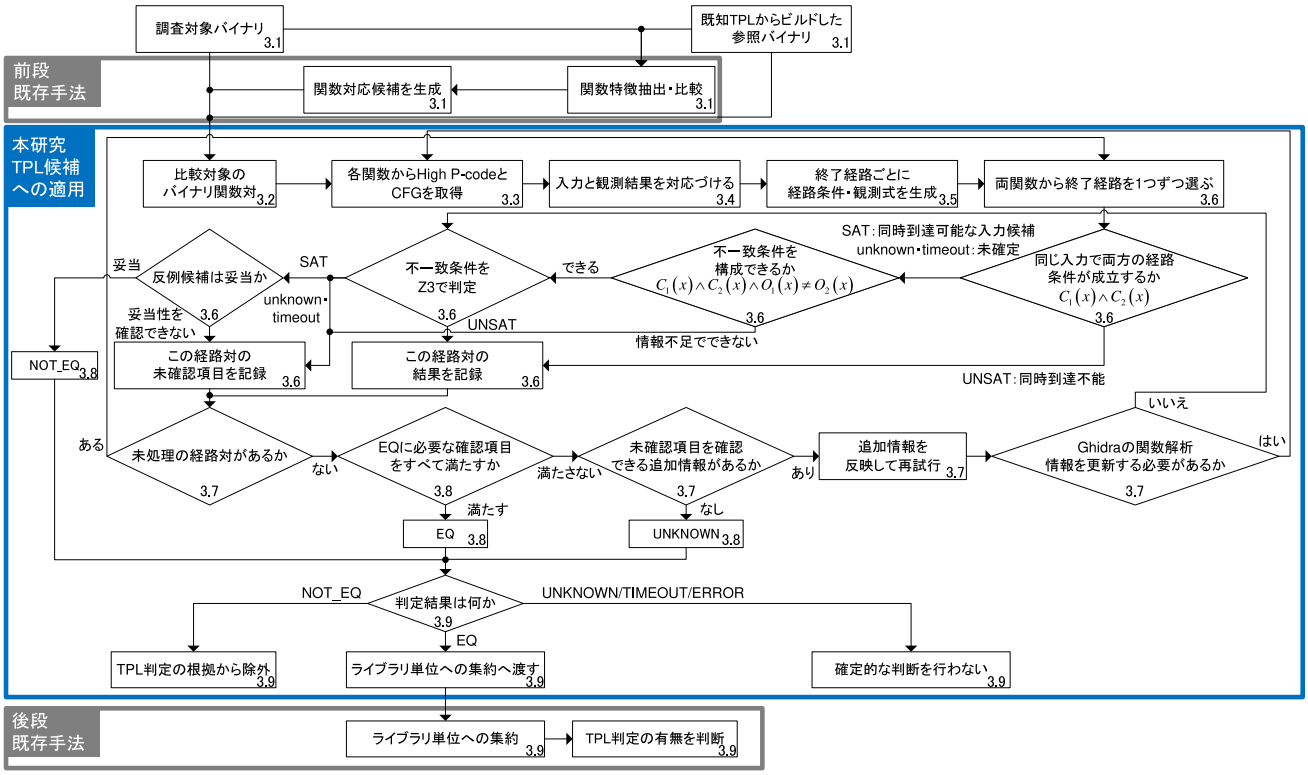


図 1　関数対の主要判定フロー
Fig. 1　Main Function-Pair Decision Flow.

の関数，f_2 を参照バイナリの関数とする．

本手法では，Application Binary Interface（ABI）に基づく格納位置と Ghidra が復元した型・格納幅を用いて入力同士を対応づける．観測結果の不一致を示す入力として得られた値を反例候補，判定に必要であるが解析によって確認できていない事項を未確認項目と呼ぶ．

提案手法は，関数対 F の解析結果を 5 種類に分類する．等価性を確認した結果を EQ，非等価性を確認した結果を NOT_EQ，判定に必要な情報を確認できなかった結果を UNKNOWN，制限時間内に判定を確定できなかった結果を TIMEOUT，解析工程を完了できなかった結果を ERROR とする．

3.3　High P-code と CFG からの情報取得

各関数の High P-code と CFG から，演算の意味，データ依存関係，および基本ブロック間の制御フローを取得する．High P-code からは，関数の戻り値を表す `RETURN`，メモリへの書き込みを表す `STORE`，関数呼出しを表す `CALL` および `CALLIND`，値の合流を表す `MULTIEQUAL` などの演算を取得する．

CFG からは，関数入口から終了点に至る終了経路を抽出する．ループを含む場合は，CFG からループを構成する基本ブロックと終了条件に関わる分岐を特定し，High P-code のデータ依存関係から反復前後の値の関係を解析する．判定に必要な情報を取得できない場合は，UNKNOWN として記録する．

3.4　入力と観測対象の対応づけ

同じ意味を持つ引数であっても，コンパイラ，最適化，アーキテクチャの違いによって，レジスタまたはスタック上での型やビット幅が異なる．本手法では，ABI に従って各引数がレジスタまたはスタック上のどの位置に格納されるかを手がかりに，対応する引数同士を特定する．

対応づけた引数のビット幅が等しい場合は，2 つのビットベクトルが等しいという制約を SMT 式に加える．異なる場合は，重なる下位ビットを等置し，残る上位ビットを自由変数として扱う．この近似は ABI 上の入力を過大に含むため，反例候補には ABI と型情報から導出した符号拡張またはゼロ拡張制約を加えて再確認し，上位ビットが符号拡張またはゼロ拡張の規則に従うことを確認できない場合は NOT_EQ の根拠にしない．1 つのレジスタに複数引数の断片がある場合は，格納位置と符号性の推定結果から拡張制約を個別に導出し，対応を特定できない断片は制約しない自由変数として扱う．

観測対象として比較するのは，関数の戻り値と，出力ポインタとして扱われる引数を介して書き込まれるメモリの内容である．比較対象とする共通の入力範囲を D_F とし，各バイナリの ABI 上の入力表現が D_F の同じ入力を表す

© 2026 Information Processing Society of Japan

よう対応づける．調査対象バイナリを $b=1$，参照バイナリを $b=2$ とし，入力 $x \in D_F$ に対するバイナリ b の観測結果を $O_b(x)$ とする．また，比較式に含めない情報が判定へ影響しないことと，対象の経路が到達可能な入力で終了することを安全条件と呼ぶ．

$$\forall x \in D_F, \quad O_1(x) = O_2(x) \tag{1}$$

本手法は，式 (1) を比較対象とした観測結果の範囲で確認し，かつ安全条件を満たした場合に，2 つの関数を等価と判定する．

戻り値は値幅と格納幅を区別し，幅が異なる場合は重なる下位ビットを比較する．等価と判定するには，除外した上位ビットが ABI 上の拡張規則に従い観測結果の差へ影響しないことを確認する．

出力ポインタ以外への副作用，すなわち volatile アクセスや I/O などは，現在の実装では $O_b(x)$ の比較式に含めない．関数の終了種別である正常終了，異常終了，トラップの区別も同様である．ただし，これらが判定へ影響しないことは，EQ または NOT_EQ を返すための安全条件として確認する．

3.5 経路条件と観測式の生成

バイナリ $b \in \{1,2\}$ の終了経路の一つを k とし，その経路条件を $C_{b,k}(x)$，観測式を $O_{b,k}(x)$ と表す．経路条件 $C_{b,k}(x)$ は，CFG 上で経路 k を構成する各分岐の条件を，その経路で選択される分岐方向に従って組み合わせることで構築する．分岐条件に含まれる値については，High P-code 上の値の定義を逆向きにたどり，入力 x から表される式へ変換する．観測式 $O_{b,k}(x)$ については，`RETURN` を起点に値の定義を逆向きにたどり，戻り値を表す式を構築する．出力ポインタへの書き込みは，2 つのバイナリで同じ意味を持つメモリ領域として対応づけられ，かつ書き込み範囲を特定できる場合にのみ観測結果として扱う．

ループは，反復前後の関係を式として要約できる場合に限り，観測結果を表す式へ組み込む．ループ内で繰り返し更新される変数について，ループ終了時の値を入力から直接表す式を構築できない場合がある．このとき，High P-code 上には `MULTIEQUAL` を介して更新前後の値が循環して参照される構造が残る．本稿では，この構造に含まれ，実際の反復回数に対応する値を確定できないものを循環値と呼ぶ．循環値や終了条件が残る経路の観測結果は式として確定できないため，当該経路の未確認項目として記録する．

3.6 経路対の比較

関数 f_1 の終了経路の一つを k，関数 f_2 の終了経路の一つを ℓ とし，その組 (k,ℓ) を経路対と呼ぶ．経路対 (k,ℓ) ごとに，まず両経路条件 $C_{1,k}(x)$ と $C_{2,\ell}(x)$ の同時成立性を Z3 で判定する．この問い合わせには経路対ごとの制限時間を設け，以下これを個別タイムアウトと呼ぶ．また，関数対 F の処理全体にも制限時間を設け，これを関数対全体の制限時間と呼ぶ．

Z3 による充足可能性判定の結果は，`sat`，`unsat`，`unknown` のいずれかとなる．`sat` は条件を満たす入力が存在すること，`unsat` は条件を満たす入力が存在しないことを表す．`unknown` は充足可能性を確定できなかったことを表す中間結果であり，それ自体は関数対の等価性または非等価性を意味しない．同じ入力で両経路条件を同時に満たせないことを同時到達不能と呼ぶ．両経路条件の同時成立性が `unsat` であれば，同時到達不能として記録する．

それ以外の経路対については，入力対応と型・幅の変換に関する制約を $A_{k\ell}(x)$，観測結果のいずれかが異なるという条件を $\Delta O_{k\ell}(x)$ とする．このとき，次の不一致条件を構成する．

$$A_{k\ell}(x) \wedge C_{1,k}(x) \wedge C_{2,\ell}(x) \wedge \Delta O_{k\ell}(x) \tag{2}$$

式 (2) が `unsat` であれば，その経路対では観測結果の差が存在しないと記録する．`sat` であれば，得られた入力を反例候補とし，それが到達可能な同一入力における観測結果の差であることを，経路，メモリ更新，到達可能な入力での終了の情報と照らして確認する．なお，ループが要約されていない場合，得られた反例候補は循環値を自由変数のまま含むため，NOT_EQ の根拠として採用できない．

不一致条件を構成できない場合，SMT 問い合わせが `unknown` となる場合，または個別タイムアウトが発生した場合は，経路対単位の未確定状態として記録し，当該経路対の判定を保留する．SMT 問い合わせが `unknown` となる場合は，その要因を確認する．タイムアウトに起因するものをタイムアウト由来の未確定状態と呼び，それ以外の場合は SMT ソルバによる充足可能性判定を確定できなかった未確認項目として扱う．これらの未確定状態は，追加情報を用いた再試行によって解消される場合があり，直ちに関数対の最終結果を決定するものではない．

3.7 未確認項目の確認と再試行

未処理の経路対が残る場合は，次の経路対について 3.6 節の比較を行う．全ての経路対を処理した後，経路対単位で記録した未確定状態と，経路条件および観測式の生成時に記録した未確認項目を確認する．追加情報によって未確認項目を解消できる場合は，必要に応じて Ghidra の関数解析情報を更新し，更新した関数解析情報を解析結果へ反映し、必要な制約を再構成したうえで判定を再試行する．

未確認項目を解消するために一時的に仮定する型を型仮説と呼ぶ．Ghidra が復元した戻り値型または引数型に不整合が疑われる場合は，Ghidra 上の関数型を永続的に変更せず，根拠に基づく型仮説を解析結果へ一時適用する．

© 2026 Information Processing Society of Japan

例えば，戻り値型が unsigned 系である一方，INT_SDIV などの符号付き演算から強い符号性の根拠を得た場合は，同じ幅の signed 系を仮説とする．

仮説を適用した解析結果から入力対応と不一致条件を再構成し，3.6 節の条件を再確認する．仮説を裏付ける根拠が十分でない場合は，EQ または NOT_EQ を確定せず，未確認項目として残す．

3.8 判定結果の集約

全ての経路対の結果と，再試行後に残った未確認項目を集約し，関数対の最終結果の詳細について述べる．

EQ 次の全てを確認した場合の結果である．(1) 入力の対応づけと値の変換が確定している．(2) 比較した経路集合が観測結果に影響する到達可能な経路を網羅している．(3) 各経路対が同時到達不能であるか，不一致条件が unsat である．(4) 除外した上位ビット，比較式に含めない副作用および終了種別が判定へ影響しない．(5) 対象の経路が到達可能な入力で終了する．

NOT_EQ 不一致条件が sat となり，得られた反例候補が到達可能な同一入力における観測結果の差であることを，値・経路・メモリ更新・終了の情報から確認できた場合の結果である．

UNKNOWN 解析は完了したが，EQ に必要な確認項目，または NOT_EQ に必要な反例候補の妥当性確認に未確認項目が残る場合の結果である．ループの要約不足，上位ビットの拡張規則，関数呼出しの効果，副作用または終了に関する確認不足に加え，Z3 の unknown のうちタイムアウト以外の理由による SMT ソルバが充足可能性を確定できなかった場合が該当する．

TIMEOUT 個別タイムアウトに由来する未確定状態が残るか，関数対全体の制限時間内に最終判定を確定できなかった場合の結果である．

ERROR デコンパイルの失敗や SMT ソルバの利用不可などにより解析工程を完了できず，意味的判定へ到達できなかった場合の結果である．

3.9 TPL 候補への適用

EQ となる関数対は，本手法の比較範囲で一致した候補としてライブラリ単位の集約へ渡す．NOT_EQ となる関数対応候補は，TPL 判定の根拠から除外する．UNKNOWN，TIMEOUT および ERROR では，確定的な判断を行わない．

4. 評価と考察

4.1 実験条件

EqBench[2] は，等価・非等価な C および Java のプログラム対を収録するデータセットである．本稿では C 版を用い，同一 C ソースから生成した関数を比較する Eq と，ソースコード上で関数の動作が異なる関数対を対象とする Neq-Core の 2 種類を使用する．Eq では，147 ソースに含まれる 411 関数について，ISA をまたぐ組合せを含む異なるビルド環境間で比較し，EQ 判定の範囲を評価する．

Neq-Core では，EqBench に収録された，動作が異なる二つの C プログラムからなるプログラム対を用いる．ただし，二つのプログラム全体の動作が異なっていても，両方に含まれる全ての関数の動作が異なるとは限らない．そこで，二つのプログラム間で実装が異なる関数，または実装が異なる関数を呼び出すことで動作に差が生じる関数を調べ，関数単位でも動作が異なる 168 関数対を評価対象として選定した．Neq-Core の目的は，異なるソースから生成された関数対の動作差をバイナリから検出し，NOT_EQ を確定できるかを評価することである．クロス ISA 比較では，ソースの違いに起因する差と ISA の違いに起因する差が判定結果に混在するため，同一 ISA 内の比較を評価対象とし，クロス ISA における非等価判定は評価範囲外とする．比較数は，Eq が 16 環境から異なる 2 環境を選ぶ 120 組／関数の計 49,320 比較，Neq-Core が一方のソースの 8 環境と他方のソースの 8 環境を総当たりする 128 組／関数対の計 21,504 比較である．

判定器への入力は 2 つのバイナリのみであり，ソースコードは評価対象の選定と正解対応の整理にのみ用いた．Clang 14，ARM64 向け GCC 11，MIPS64 向け GCC 10 でビルドし，Ghidra 12.0.1 と Z3 4.16.0 で解析した．制限時間は，SMT 問い合わせ 10 秒，同時成立性判定 5 秒，関数対全体 600 秒とし，Intel Core i7-14700K，メモリ 125 GiB の Ubuntu 24.04.4 LTS 環境で実行した．

4.2 研究課題

本章では，次の三つの研究課題を扱う．

RQ1 ERROR を含む全比較を分母としたとき，Eq における EQ 判定率と，Neq-Core における NOT_EQ 検出率はどの程度か．

RQ2 意味的判定に到達した EQ・NOT_EQ・UNKNOWN・TIMEOUT は，反復，戻り値幅，関数呼出しなどのプログラム特徴とどのように関連するか．

RQ3 5 種類の最終結果に至った事例から判定の過程と限界を示すとともに，ERROR については意味的判定へ到達できなかった原因を分析する．

4.3 全体結果（RQ1）

Eq と Neq-Core は対象と組合せ規則が異なるため，最終結果を分けて**表 1** に示す．

Eq の全 49,320 比較のうち、EQ となる比較は 12.9%の 6,365 比較であり，1 比較以上で EQ を確定した関数は 43.6%の 179 ／ 411 関数である．正解ラベルと矛盾する NOT_EQ はなかったため，本評価範囲では，同一の動作をする関数

© 2026 Information Processing Society of Japan

表 1　Eq および Neq-Core の全体結果と主要環境条件別の確定率
Table 1　Overall Results and Decision Rates under Major Environment Conditions.

(a) Eq

EQ	NOT_EQ	UNKNOWN	TIMEOUT	ERROR	合計
6,365	0	23,999	18,794	162	49,320

1 比較以上で EQ を確定した関数数　179 ／ 411（43.6%）

(b) Neq-Core

EQ	NOT_EQ	UNKNOWN	TIMEOUT	ERROR	合計
192	1,538	6,478	13,204	92	21,504

比較単位の NOT_EQ 判定割合　7.2%（1,538 ／ 21,504）
NOT_EQ を 1 組以上確定した関数対数　28 ／ 168（16.7%）

(c) 主要環境条件別の確定率

評価	条件	比較数（同一／相違）	確定率（同一／相違）
Eq	ISA	23,016 ／ 26,304	EQ 18.4 ／ 8.1%
Eq	コンパイラ	23,016 ／ 26,304	EQ 13.1 ／ 12.7%
Eq	最適化	9,864 ／ 39,456	EQ 12.4 ／ 13.0%
Neq-Core	コンパイラ	10,752 ／ 10,752	NOT_EQ 7.5 ／ 6.8%
Neq-Core	最適化	5,376 ／ 16,128	NOT_EQ 8.4 ／ 6.7%

表 2　ソース上のプログラム特徴と最終結果の関連
Table 2　Association between Source-Level Program Features and Outcomes.

評価	特徴層	比較数	確定率	UNKNOWN	TIMEOUT
Eq	反復あり	13,800	3.9%	32.7%	63.4%
Eq	反復なし	35,520	16.4%	54.9%	28.3%
Eq	呼出しあり	26,040	2.0%	31.0%	66.8%
Eq	呼出しなし	23,280	25.1%	68.5%	6.0%
Eq	分岐箇所 0	17,640	21.0%	66.6%	11.9%
Eq	分岐箇所 1	15,600	14.8%	51.6%	33.6%
Eq	分岐箇所 2–3	9,840	3.4%	35.3%	60.7%
Eq	分岐箇所 4 以上	6,240	0.4%	11.4%	87.9%
Neq-Core	反復あり	8,704	4.7%	26.7%	68.6%
Neq-Core	反復なし	12,800	8.8%	32.5%	56.5%
Neq-Core	呼出しあり	16,128	1.0%	21.3%	77.5%
Neq-Core	呼出しなし	5,376	25.7%	56.6%	13.3%
Neq-Core	分岐箇所 0	3,968	16.7%	50.2%	27.2%
Neq-Core	分岐箇所 1	7,040	6.5%	43.7%	49.6%
Neq-Core	分岐箇所 2–3	5,888	6.1%	20.0%	73.5%
Neq-Core	分岐箇所 4 以上	4,608	1.4%	5.2%	93.4%

対を誤って NOT_EQ と判定し，TPL 判定の根拠から除外するケースはない．一方，UNKNOWN が 48.7%の 23,999 比較，TIMEOUT が 38.1%の 18,794 比較を占め，同じ関数でもビルド環境によって EQ を確定できる範囲は限定的である．Neq-Core では，7.2%の NOT_EQ は 1,538 比較であり，少なくとも 1 比較で差を確定できたのは 16.7%の 28 関数対である．したがって，後段検証器として確実に除外できた偽候補は一部であり，残る UNKNOWN，TIMEOUT および ERROR を保守的に候補として残すなら除去効果は限定的である．

主要環境条件別の結果を表 1(c) に示す．Eq では ISA 相違時に EQ 率が低下した．コンパイラ相違による低下は小さく，最適化レベル相違では Eq の EQ 率低下は見られず，Neq-Core の NOT_EQ 率はやや低下した．

以後，正解ラベルと矛盾して EQ と判定された結果を偽 EQ と呼ぶ．Neq-Core の EQ192 件は偽 EQ であり，原因を RQ3 で分析する．また，Eq の EQ 結果にも，同種の観測漏れが残る例がある．具体的には，両バイナリの High P-code にメモリへの書き込みを表す `STORE` 演算があるにもかかわらず，書き込み先の内容が比較式に含まれない比較が 18 関数 504 件に現れた．Eq では正解ラベル上等価であるため偽 EQ とは断定できないが，EQ 出力をそのまま健全な等価証拠とはみなせない．

計算時間は結果により大きく異なる．Eq のペア実行記録から得た中央値（第 1–第 3 四分位）は，EQ が 34.8 秒（26.5–45.0 秒），UNKNOWN が 61.0 秒（39.4–105.7 秒），TIMEOUT が 602.9 秒（602.2–604.2 秒），ERROR が 31.1 秒（29.2–85.7 秒）である．Neq-Core で終端時刻まで記録された NOT_EQ 1,538 件の中央値は 15.6 秒（11.3–33.7 秒）であり，差を示す一つの反例で確定できる場合は比較的短い．一方，Eq では全比較の中央値が 110.6 秒，平均が 288.0 秒であり，数万候補へ適用する後段検証器としては，並列実行を用いても計算コストが大きい．

4.4　プログラム特徴との関連（RQ2）

プログラム特徴は，評価用 C ソースから，反復文，関数呼出し，および分岐構文の有無を確認し，**表 2** に主要な層別結果を示す．確定率は，Eq では EQ 率，Neq-Core では NOT_EQ 率である．Eq の NOT_EQ は全層で 0.0%である．

表 2 のとおり，Eq と Neq-Core のいずれでも，反復または呼出しを含む関数，および分岐箇所が多い関数ほど確定率が低下し TIMEOUT 率が上昇した．また分岐箇所が増えると，未確定結果は UNKNOWN より TIMEOUT へ移る．すなわち，本実装では，反復，呼出しおよび経路分岐が単に判定不能理由を増やすだけでなく，600 秒以内に証明探索を終えられるかを強く制約している．ただし，これらの特徴は相互に相関し，同一関数から多数の環境対を生成しているため，割合を各特徴の独立な因果効果とは解釈しない．

反復と呼出しの交絡へ対処するため，両特徴を同時に層別した結果を**表 3** に示す．表 3 のとおり，Eq と Neq-Core のいずれでも，反復の有無によらず呼出しがあると TIMEOUT 率が大きく増加した．一方，呼出しなしの層では反復により TIMEOUT 率が増えるため，反復も独立した追加負荷を持つと考えられる．二特徴の範囲では，呼出しがより大きな律速として残り，反復と呼出しが重なる場合に最も深刻である．ただし，分岐数や関数規模を同時に固定

© 2026 Information Processing Society of Japan

表 3　反復と呼出しの同時層別
Table 3　Joint Stratification by Loops and Calls.

評価	反復	呼出し	確定率	TIMEOUT 率
Eq	なし	なし	EQ 28.6%	1.2%
Eq	なし	あり	EQ 3.1%	58.0%
Eq	あり	なし	EQ 11.4%	25.3%
Eq	あり	あり	EQ 0.0%	83.0%
Neq-Core	なし	なし	NOT_EQ 36.3%	10.6%
Neq-Core	なし	あり	NOT_EQ 1.5%	68.7%
Neq-Core	あり	なし	NOT_EQ 15.2%	15.9%
Neq-Core	あり	あり	NOT_EQ 0.0%	92.2%

していないため，これは因果効果の完全な分離ではない．

UNKNOWN の未充足理由は強く共起した．Eq 23,999 件では経路完全性不足が 9,977 件，戻り値幅の由来不足が 8,223 件，反例安全性不足が 7,595 件，呼出し対応不足が 4,409 件に現れ，Neq-Core 6,478 件でも経路完全性不足 4,336 件などが多いため，理由の出現数を個別に足して改善可能件数とみなせない．Eq で他の理由を伴わない比較は戻り値幅の由来不足 3,617 件が最大で，呼出し対応のみは 52 件にすぎない．したがって，まず EQ の健全性に直結する出力メモリ観測を修正し，次に戻り値幅を扱い，その後に経路・呼出し・反例安全性を組み合わせて改善する必要がある．

制限時間感度を調べるため，TIMEOUT から偏りが生じないよう各 TIMEOUT ペア集合の 1%にあたる Eq 188 件，Neq-Core 132 件の計 320 件を抽出し，関数対上限を 600 秒から 900 秒へ延長したところ，320 ／ 320 件が 900 秒でも TIMEOUT となり，確定結果への遷移は 0 件である．少なくとも無作為標本では，改善しなかった．資源配分だけでなく，大規模な関数の経路・呼出し処理そのものを改善する必要がある．

4.5　結果別の判定事例（RQ3）

EQ，NOT_EQ，UNKNOWN，TIMEOUT，ERROR の 5 種類の最終結果について，代表的な事例と，その最終結果に至った要因を分析する．また，Neq-Core で確認された EQ の誤判定についても，その原因を分析する．

4.5.1　反復処理を含む関数の判定事例

Eq に含まれる `REVE/barthe2/f` は，正の入力 n について 1 から n までを加算し，非正入力では 0 を返す．ARM64・Clang・O0 版では，High P-code に反復ごとに更新される `MULTIEQUAL` 由来の循環値が残る一方，O1 版では入力 n から戻り値を計算できる閉じた式となる．

O0 版の循環値は反復前後の関係へ結び付いていないため，自由変数として不一致条件へ現れ，Z3 は `sat` となる割当を得られる．しかし，その値が実際のループ実行で到達可能かは確認できず，`sat` だけでは NOT_EQ の根拠にならない．これは 3.6 節で述べたとおりである．本事例では，ループの要約と反例候補の妥当性確認が制限時間内に完了せず，最終結果は TIMEOUT となる．同じ関数でも，MIPS64・Clang・O2 ／ O3 版では全不一致条件を `unsat` と確認して EQ となり，循環値が残る ARM64・Clang・O0 ／ MIPS64・GCC・O0 版では到達可能な入力での終了を確認できず UNKNOWN となる．

4.5.2　戻り値幅の判定事例

Eq に含まれる `ltfive/lib` の ARM64・Clang・O0 版は戻り値の格納幅が 32 ビット，MIPS64・GCC・O1 版は 64 ビットである．重なる下位 32 ビットは比較できるが，64 ビット側の上位ビットが正しい符号拡張であり，観測結果の差へ影響しないことを確認できなかった．上位ビットを除外したままでは EQ の条件を満たさず，下位ビットから妥当な反例も確認できなかったため，最終結果は UNKNOWN となる．

4.5.3　NOT_EQ の判定事例

Neq-Core に含まれる `airy/airy/MAX` では，入力 a, b に対し，$a < b$ が偽のとき両関数とも a を返すが，真のとき一方は 0，他方は b を返す．対応入力と両関数の経路条件に $0 \neq b$ を加えた不一致条件は `sat` となり，得られた入力が到達可能な同一入力における実際の戻り値差であることを確認した．4 経路対のうち 3 件は `unsat`，1 件は妥当な反例であるため，最終結果は NOT_EQ となる．

Neq-Core の 168 関数対を動作差の種類で重複分類すると，データ・算術式の差では 22 ／ 89 関数対で NOT_EQ を確認した一方，述語・制御や反復・再帰では 1 割前後，呼出し・戻り値・I/O では 0 件である．この結果は，局所的な算術差を直接表現できる場合に検出しやすく，制御・反復・呼出しを介する差では NOT_EQ の確定が難しいことを示す．

4.5.4　偽 EQ の原因

Neq-Core で偽 EQ となる 192 件は，`raytrace` に含まれる 3 関数の MIPS64 比較で発生した．これらの関数は構造体を返す関数であり，MIPS64 版では構造体の返却先が暗黙のポインタ引数として扱われていた．本実装は返却先ポインタを対応づけていた一方，そのポインタを介して構造体へ書き込まれる値を観測結果に含めていなかった．そのため，実際には構造体の内容に差があっても不一致条件にその差が現れず，`unsat` となって EQ と判定された．したがって，192 件の偽 EQ は，構造体返却に伴うメモリ更新を観測結果へ反映できていなかったことが原因である．構造体を返す関数では，返却先ポインタだけでなく，そのポインタを介して書き込まれるメモリ内容を比較する必要がある．

4.5.5　ERROR の原因

ERROR は Eq で 162 件，Neq-Core で 92 件である．Eq では 135 件が未宣言のスタック入力等を含む不正な SMT 式，18 件がビットベクトル幅の不一致，9 件が終端結果の

© 2026 Information Processing Society of Japan

整合性検査失敗である．Neq-Core では 67 件が未宣言シンボルを含む SMT 式，25 件が条件式の正規化時に整数と欠測値を比較したことを起点とする終端結果の整合性検査失敗である．いずれもプログラムの等価性を示す証拠ではなく，式の構成または結果確定処理の失敗として ERROR に分離されている．

4.6 妥当性に関する留意点

第一に，判定は ABI および Ghidra の型，値幅，High P-code 復元にも依存する．

第二に，RQ2 で述べたとおり，制限時間の設定は UNKNOWN と TIMEOUT の分布に影響し得る．

第三に，C 言語で未定義動作となる入力では，同じソースから生成したバイナリであっても動作が一致するとは限らない．したがって，EqBench の正解ラベルとの対応は，C 言語の動作が定義される入力の範囲で解釈する．

第四に，NOT_EQ の反例は構築した式上で確認したものであり，独立した具体実行による全件再現は行っていない．

5. おわりに

本稿では，TPL 検出器等が生成した関数対応候補を，ライブラリ単位への集約前に，対応する入力に対する観測結果に基づいて再検証する手法を提案した．Ghidra の High P-code と SMT ソルバ Z3 を用い，対象の 2 つのバイナリについて抽出した戻り値と出力メモリを比較する．

評価では EqBench の Eq と Neq-Core を集計した．Eq では一定数の EQ を得て，正解と矛盾する NOT_EQ はなかった．主要環境条件別には，ISA 相違時に EQ 率が低下した一方，最適化レベル相違による Eq の EQ 率低下は見られなかった．Neq-Core では NOT_EQ を得たが，検出範囲は一部の関数対に限られ，構造体返却先のメモリ観測漏れにより偽 EQ が生じた．また，反復，呼出し，分岐数の増大は TIMEOUT 率と強く関連した．

TPL 検出パイプラインでは，NOT_EQ のみを候補除外に用いる保守的な運用により，本評価範囲では真の対応候補を誤って除外しなかった．ただし，除外できた偽候補は一部の関数対に限られ，UNKNOWN，TIMEOUT，ERROR を非等価の代用にすると真の候補を多数失う．したがって現時点の価値は，一部の偽候補へ安全な除外根拠を追加できる点に限られる．

他方，EQ を候補の確証として利用するには設計上の制約が残る．偽 EQ と `STORE` 観測漏れの事例が示す教訓は，個別の SAT・UNSAT は途中結果にすぎず，EQ の健全性は観測式が必要な出力を全て被覆していることの確認に依存するという点である．

今後は，優先度の第一を，隠れた構造体返却先を含む ABI 上の出力領域の完全な追跡とし，必要なメモリ効果の包含を EQ 昇格の機械検査可能な前提にする．次に，Eq で単独理由が最多であった戻り値幅の由来を扱い，未宣言の生存入力とビット幅不一致を解消する．その後，共起の多い経路完全性，呼出し効果および反例安全性を一体として改善し，反復要約と経路探索の計算量を削減し，最後に TPL 検出器へ接続してライブラリ単位の適合率・再現率を評価する．

参考文献

[1] Jiang, L., An, J., Huang, H., Tang, Q., Nie, S., Wu, S., and Zhang, Y.: BinaryAI: Binary Software Composition Analysis via Intelligent Binary Source Code Matching, Proc. IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp.1–13 (2024).

[2] Badihi, S., Li, Y. and Rubin, J.: EqBench: A Dataset of Equivalent and Non-Equivalent Program Pairs, Proc. 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR), pp.610–614 (2021).

[3] Yang, C., Xu, Z., Chen, H., Liu, Y., Gong, X., and Liu, B.: ModX: Binary Level Partially Imported Third-Party Library Detection via Program Modularization and Semantic Matching, Proc. 44th International Conference on Software Engineering (ICSE), pp.1393–1405 (2022).

[4] Gao, D., Reiter, M. K. and Song, D.: BinHunt: Automatically Finding Semantic Differences in Binary Programs, Proc. International Conference on Information and Communications Security (ICICS), pp.238–255 (2008).

[5] Ming, J., Pan, M. and Gao, D.: iBinHunt: Binary Hunting with Inter-Procedural Control Flow, Proc. ICISC, LNCS 7839, pp.92–109 (2013).

[6] He, X., Wang, S., Feng, P., Wang, X., Sun, S., Li, Q., and Sun, K.: BinGo: Identifying Security Patches in Binary Code with Graph Representation Learning, Proc. 19th ACM Asia Conference on Computer and Communications Security, pp.1186–1199 (2024).

[7] Egele, M., Woo, M., Chapman, P., and Brumley, D.: Blanket Execution: Dynamic Similarity Testing for Program Binaries and Components, Proc. 23rd USENIX Security Symposium (USENIX Security 14), pp.303–317 (2014).

[8] Ming, J., Xu, D., Jiang, Y., and Wu, D.: BinSim: Trace-Based Semantic Binary Diffing via System Call Sliced Segment Equivalence Checking, Proc. 26th USENIX Security Symposium (USENIX Security 17), pp.253–270 (2017).

[9] Sharma, R., Schkufza, E., Churchill, B., and Aiken, A.: Data-Driven Equivalence Checking, Proc. 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, pp.391–406 (2013).

[10] Putro, P. A.: Low-Level Code Semantic Equivalence Checker, Programming and Computer Software, No. 6, pp.53–71 (2025). DOI: 10.7868/S3034584725060047.

[11] Naus, N., Verbeek, F., Walker, D., and Ravindran, B.: A Formal Semantics for P-Code, Proc. Working Conference on Verified Software: Theories, Tools, and Experiments, pp.111–128 (2022).

© 2026 Information Processing Society of Japan